

Request for Quotation: AI Search Infrastructure (ETL Pipeline + Search API)

Project Overview

We require a complete AI search infrastructure with **bilingual support (Hungarian + English)** consisting of:

1. **ETL Pipeline:** Process 38,000+ B2B categories and 5,000-10,000 company profiles
2. **Search Microservice:** REST API providing hybrid search (Vector + BM25 with RRF scoring)

Core Technologies: Python 3.10+, PostgreSQL + pgvector, OpenAI (text-embedding-3-small, gpt-4o-mini), huspacy, spacy, FastAPI/Flask

Part 1: ETL Pipeline (Data Processing)

1.1 Database Schema Implementation

You must implement the following PostgreSQL schema with **bilingual support**:

category_vectors table:

```
SQL
- category_id (FK)
- language (VARCHAR 2) - 'hu' or 'en'
- content_text (TEXT) - complete narrative text
- embedding (vector 1536)
- bm25_tokens_hu (tsvector) - Hungarian full-text index
- bm25_tokens_en (tsvector) - English full-text index
- metadata (JSONB): ai_type, status, hirdetesi_kulcsszavak,
                    szintetikus_kerdesek, temakori_javaslatok
- content_hash (VARCHAR 32)
- updated_at (TIMESTAMP)
```

Required Indexes:

- pgvector index on **embedding** (IVFFlat or HNSW) - language-agnostic
- GIN index on **bm25_tokens_hu**
- GIN index on **bm25_tokens_en**
- GIN index on **metadata**
- B-tree index on **category_id**
- B-tree index on **language**

company_vectors table:

SQL

```
- company_id (FK)
- language (VARCHAR 2) - 'hu' or 'en'
- chunk_index (INTEGER)
- embedding (vector 1536)
- content_chunk (TEXT) - with context header
- bm25_tokens_hu (tsvector)
- bm25_tokens_en (tsvector)
- metadata (JSONB): company_id, company_name, chunk_index,
                    total_chunks, category,
synthetic_questions,
                    source_type, product_id
- created_at (TIMESTAMP)
```

Required Indexes: Same pattern as category_vectors + B-tree on **company_id**, **language**

1.2 Multi-Language Processing Strategy

Dual Processing Approach:

Each category and company is processed in **BOTH languages**:

1. **Hungarian version** (primary - if data is in Hungarian)
2. **English version** (translation or native English data)

Language Detection & Routing:

- Auto-detect source language (langdetect library)
- If Hungarian source → process Hungarian + translate to English
- If English source → process English + translate to Hungarian
- If bilingual source → process both natively

Translation Service:

- Use OpenAI gpt-4o-mini for high-quality translation
- Preserve technical terms, brand names
- Cost: ~\$0.00001 per category translation

1.3 Pipeline A: Category Processing (38,000+ categories × 2 languages)

Required Steps (per language):

1. **Hash-Based Change Detection** - Separate hash per language
2. **Hierarchical Batch Building** - Max 25 categories/batch
3. **AI Enrichment** - Language-aware prompts

AI Enrichment (bilingual):

None

Hungarian prompt: "Bemenetként kapsz kategória neveket..."
English prompt: "You receive category names and descriptions..."

Generate for each language:

- **ai_type**: Same classification in both languages
 - **hirdetesi_kulcsszavak**: Language-specific keywords
 - **szintetikus_kerdesek**: Language-specific questions
4. **Optimized Prompt Engineering** - 97%+ per language
 5. **Quality Check & Retry** - Per language validation
 6. **Duplicate Detection** - Cross-language duplicate check
 7. **Filtering** - DELETE status applies to both languages
 8. **Narrative Template Building** - Language-specific templates

Hungarian Template:

None

"{category_name} kategória, amely {type} csoportjába tartozik.
Ismeret még mint: {synonyms}.
Részterület: {hierarchy_path}.
Kapcsolódó fogalmak: {keywords}.
Tipikus felhasználói kérdések: {questions}.
{description}"

English Template:

None

```
"{category_name} category, which belongs to the {type} group.  
Also known as: {synonyms}.  
Area: {hierarchy_path}.  
Related concepts: {keywords}.  
Typical user questions: {questions}.  
{description}"
```

9.

Parallel Vectorization (per language)

- Same narrative → OpenAI embedding (multilingual model)
- Hungarian text → huspacy lemmatization → bm25_tokens_hu
- English text → spacy lemmatization → bm25_tokens_en

10. **PostgreSQL Insert** - Two rows per category (one per language)

1.4 Pipeline B: Company Processing (5,000-10,000 companies × 2 languages)

Required Steps (bilingual):

1. **Language Detection** - Identify source language
2. **Translation** - Generate missing language version
3. **Hash-Based Change Detection** - Per language
4. **Narrative Data Assembly** - Language-specific templates

Hungarian Assembly:

None

```
"{company_name} egy {category} profillal rendelkező cég.  
{description}  
Termékek: {products with properties}  
Blog: {blog summaries}"
```

English Assembly:

None

```
"{company_name} is a company with {category} profile."
```

```
{description}  
Products: {products with properties}  
Blog: {blog summaries}"
```

5. **HTML Cleaning** - Same for both
6. **AI Synthetic Questions** - Language-specific

Hungarian questions: "Vállalnak-e komplett raktáratomatizálást?"

English questions: "Do you provide complete warehouse automation?"

7. **Smart Chunking** - Context header in both languages

Hungarian context:

```
None  
[CÉG: {company_name}] [PROFIL: {category}]  
{chunk content}
```

English context:

```
None  
[COMPANY: {company_name}] [PROFILE: {category}]  
{chunk content}
```

8-9. **Per-Chunk Vectorization** - Dual BM25 tokens per chunk 10. **Metadata Building** - Language-agnostic JSONB 11. **Atomic Swap** - Delete/Insert per language

1.5 NLP Library Requirements

Hungarian Processing:

```
Python  
import huspacy  
nlp_hu = huspacy.load("hu_core_news_lg")  
  
def lemmatize_hungarian(text):
```

```
doc = nlp_hu(text)
return [token.lemma_.lower() for token in doc if
token.is_alpha]
```

English Processing:

```
Python
import spacy
nlp_en = spacy.load("en_core_web_lg")

def lemmatize_english(text):
    doc = nlp_en(text)
    return [token.lemma_.lower() for token in doc if
token.is_alpha]
```

PostgreSQL Configuration:

```
SQL
-- Hungarian index
CREATE INDEX idx_bm25_hu ON category_vectors
USING gin(to_tsvector('hungarian', content_text));

-- English index
CREATE INDEX idx_bm25_en ON category_vectors
USING gin(to_tsvector('english', content_text));
```

Part 2: Search Microservice (REST API)

2.1 API Architecture

Language Parameter: All endpoints accept optional `language` parameter (default: auto-detect)

```
JSON
{
```

```
"query": "electric forklift rental",
"language": "en", // 'hu' or 'en' or 'auto'
"limit": 5
}
```

2.2 Endpoint 1: Category Search (Bilingual)

Endpoint: `POST /search/category`

Language Handling:

1. Language Detection:

- If `language` parameter provided → use it
- If not → auto-detect from query text (langdetect)
- Example: "elektromos targonca" → detected as 'hu'

2. Language-Specific Search:

- Query embedding (multilingual, works for both)
- BM25 search on appropriate language index:
 - Hungarian query → search `bm25_tokens_hu`
 - English query → search `bm25_tokens_en`

3. Cross-Language Fallback:

- If results < threshold in detected language
- Try other language as fallback
- Useful for brand names, technical terms

Request Examples:

Hungarian:

```
JSON
{
  "query": "elektromos targonca bérlet",
  "language": "hu",
  "limit": 5
}
```

English:

JSON

```
{
  "query": "electric forklift rental",
  "language": "en",
  "limit": 5
}
```

Auto-detect:

JSON

```
{
  "query": "Bosch drill",
  "limit": 5 // system detects 'en'
}
```

Response Format:

JSON

```
{
  "results": [
    {
      "id": 123,
      "name": "Elektromos targonca / Electric Forklift",
      "score": 0.95,
      "language": "hu",
      "ai_type": "GÉP_ÉS_BERENDEZÉS"
    }
  ],
  "detected_language": "hu",
  "response_time_ms": 342
}
```

2.3 Endpoint 2: Company Matching (Bilingual)

Endpoint: POST /match/companies

Language Handling:

Similar to category search:

- Detect RFQ language
- Search appropriate BM25 index
- Cross-language fallback if needed

Request Example (Hungarian):

JSON

```
{
  "rfq_text": "5 db Bunting saválló kesztyű kellene, darus autóval szállítva Budapest 10. kerületbe",
  "language": "hu",
  "category_id": 123,
  "filters": {
    "location": ["Budapest"],
    "logistics": ["crane_truck"]
  }
}
```

Request Example (English):

JSON

```
{
  "rfq_text": "Need 5 Bunting acid-resistant gloves, delivery with crane truck to Budapest district 10",
  "language": "en",
  "category_id": 123,
  "filters": {
    "location": ["Budapest"],
    "logistics": ["crane_truck"]
  }
}
```

Response includes language info:

JSON

```
{
  "matches": [
    {
      "company_id": 998,
```

```

        "score": 0.95,
        "language": "hu",
        "is_highlighted": true
    }
],
    "detected_language": "hu",
    "response_time_ms": 2341
}

```

2.4 RRF Scoring (Language-Aware)

Modified RRF for Bilingual:

```

Python
def rrf_bilingual(vector_results, bm25_hu_results,
bm25_en_results,
                    query_language, k=60):
    scores = {}

    # Vector results (language-agnostic)
    for rank, result in enumerate(vector_results, 1):
        scores[result.id] = scores.get(result.id, 0) + 1/(k +
rank)

    # Primary language BM25
    primary_bm25 = bm25_hu_results if query_language == 'hu'
else bm25_en_results
    for rank, result in enumerate(primary_bm25, 1):
        scores[result.id] = scores.get(result.id, 0) + 1/(k +
rank)

    # Optional: Secondary language BM25 (lower weight)
    secondary_bm25 = bm25_en_results if query_language == 'hu'
else bm25_hu_results
    for rank, result in enumerate(secondary_bm25, 1):
        scores[result.id] = scores.get(result.id, 0) + 0.5/(k
+ rank) # 50% weight

```

```
return sorted(scores.items(), key=lambda x: x[1],
reverse=True)
```

Deliverables

Code & Infrastructure:

1. ETL Pipeline (Python 3.10+)

- Bilingual processing for Pipeline A & B
- Language detection (langdetect)
- Translation service integration
- Dual NLP libraries (huspacy + spacy)
- Docker container

2. Search Microservice (Python 3.10+)

- Language parameter support
- Auto-detection capability
- Cross-language fallback
- 2 endpoints with bilingual support
- Docker container

3. Docker Compose Package

- PostgreSQL + pgvector
- ETL service
- Search API service
- `.env.example`
- One-command startup

4. Documentation

- README.md
- API documentation (Swagger/OpenAPI) with language examples
- Bilingual processing guide
- Database schema documentation

5. Testing Suite

- Tests for both languages
 - Cross-language tests
 - Language detection tests
-

Acceptance Criteria

ETL Pipeline Tests:

Test	Requirement	Pass Criteria
Hash Check	Per language	0 API calls per language on rerun
Dual Language	Category in both	Hungarian + English rows created
Translation Quality	Sample check	Correct technical terms preserved
Language Detection	100 samples	>95% accuracy
Quality Rate	Both languages	97-99% success each

Search API Tests:

Test	Requirement	Pass Criteria
Hungarian Query	"elektromos targonca"	Correct results <500ms
English Query	"electric forklift"	Correct results <500ms
Auto-Detection	Mixed queries	>95% correct language
Cross-Language	Brand names	Finds in both languages
Bilingual Results	Same category	Both language versions returned

Cost & Performance Targets

First Full ETL Run (Bilingual):

- Categories: $\sim \$7.83 \times 2 = \sim \mathbf{\$15.66}$ (processing both languages)
- Translation: $\sim \$0.38$ (38K categories)
- Companies: $\sim \$7.50 \times 2 = \sim \mathbf{\$15.00}$
- **Total: $\sim \mathbf{\$31.04}$** (first run with translation)

Incremental Runs (Daily):

- $\sim \$1.00$ - 2.60 /day (both languages)
- **Monthly: $\mathbf{\$30-80}$**

Search API Performance:

- Both languages: <500ms (category), <3s (company)

Expected Accuracy:

- ETL processing: 97-99% per language
 - Translation quality: >95% (technical term preservation)
 - Language detection: >95%
 - Search accuracy: 93-96% per language
-

Pricing Request Format

Item	Description
ETL Pipeline	
Search Microservice	
TOTAL	

Timeline

7 weeks from contract.

Weekly: Progress report via email/Jira

Critical Constraints (Non-Negotiable)

ETL Requirements:

1. **Dual Language Support:** Every category/company in BOTH languages
2. **Narrative Templates:** Language-specific natural sentences
3. **Translation Quality:** Technical terms preserved >95%
4. **Same Vector:** Multilingual embedding model
5. **Dual BM25:** Separate indexes per language
6. **Quality Target:** 97%+ per language

Search API Requirements:

7. **Auto-Detection:** >95% accuracy
8. **Cross-Language Fallback:** Implemented
9. **RRF Bilingual:** Language-aware scoring

10. **Response Time:** Same targets for both languages

11. **Language Parameter:** Supported in all endpoints

Submission Requirements

Deadline: [DATE]

Required Documents:

1. **Pricing breakdown** (use format above)
2. **Technical approach** (max 2 pages):
 - Bilingual processing strategy
 - Translation service approach
 - Language detection methodology
 - Dual-index architecture
 - Cross-language search strategy

Contact: [Your Email]

**Based on: MODUL 2 ETL Pipeline + Search Microservice Specifications v2025-01-22
(Bilingual Extension)**